

Diagnosing and Automating the Provider API Fix

Problem: A Kubernetes persistent-volume operation failed with an HTTP 400 response from the cloud provider, because Brudo's code didn't yet know the provider's exact API schema. The goal is to make Brudo discover the correct request format itself instead of relying on Codex or manual fixes. We follow the research plan step by step.

1. Inspect the HTTP 400 Error in Detail

A 400 Bad Request means the *client's request* is malformed or invalid [15†L188-L191]. For example, an Azure PV issue showed a 400 error because "one of the HTTP headers is not in the correct format" [1†L239-L247]. So first we must capture and parse the provider's error message and logs. Common causes include missing required fields, wrong JSON schema, or invalid parameter values. As MDN notes, a 400 indicates the server "would not process the request due to something the server considered a client error" [15†L188-L191]. By examining the error body, status code, and any provided "error" or "message" fields, we pinpoint which part of the request violated the provider's contract (schema). This narrows down what Brudo must change.

2. Query the Provider's API Schemas/Endpoints

Next, we treat the cloud provider as the authority on its current API shape. Using a headless browser or API client, retrieve the official API specification or capability endpoints. For example, many cloud providers have REST endpoints like `/v1/volumes` or CLI commands (e.g. `az disk show` or `aws ec2 describe-volumes`) that reveal what operations and parameters are supported. By querying these (potentially undocumented) endpoints, Brudo can learn the required fields or versions. This aligns with the agent-first principle of "strict boundaries and predictable structure" [8†L383-L392]: we formalize the provider boundary in code. In practice we might fetch the provider's swagger schema or issue a harmless `GET` / `HEAD` on the failing operation path to see expected parameters. Codex used web search to guess the API, but now Brudo should hit a well-known "list" or "describe" endpoint and read the JSON schema directly. For example, Brudo can request the current StorageClass or Disk details from the Kubernetes/Cloud API and parse out allowed fields, ensuring the local model matches reality.

3. Craft a Minimal Diagnostic Request

With the error information and documentation, we systematically refine our request. Start with the minimal payload and gradually add fields until the call succeeds (or fully replicate the expected schema). For instance, if a volume creation failed, try removing optional fields (e.g. size, region, attachment tags) one by one. Each attempt is "discriminating": it tests which field was causing the 400. This binary-search approach is akin to the prompt's "minimal discriminating provider query." By logging each attempt and response, Brudo identifies exactly which parameter or header was unexpected. This process turns the wild-guessing step Codex did into a reproducible procedure. In effect, we ask: "Which part of our request must be changed to satisfy the provider's requirements?" Then we isolate that part.

4. Automate Provider Shape Discovery in Brudo

Once we know how to construct a valid request, we embed that logic into Brudo. We write a deterministic routine (“**provider-shape discovery**”) that, given an operation type (e.g. CreateVolume) and context (existing cluster state), queries the provider’s endpoints to auto-populate the needed fields. For example, Brudo’s code might call an endpoint like `/regions/<region>/gpu-parameters` or use the cloud SDK to fetch any service limitations. This replaces the old hard-coded “fix” messages. The code can be validated against the actual provider metadata at runtime: if the provider adds a new required flag in the future, Brudo’s discovery routine will notice the mismatch and adjust. This enforces the invariant from OpenAI’s harness engineering: the agent must “parse data shapes at the boundary” [8†L379-L382], meaning the code must explicitly handle input/output schemas. By making provider introspection part of Brudo’s pipelines (for example, refreshing local state after any 400 error), we ensure the deployment logic remains consistent with the real world.

5. Replace Codex-Dependent Repair and Validate

Finally, we remove any remaining Codex or “queued fix” steps related to this issue and rely solely on the new machinery. To validate the solution, we run a full deployment: create the persistent volume, schedule the GPU pod, and observe if Brudo successfully attaches it and runs the job. If the new provider-discovery code is correct, the previous HTTP 400 should not reappear. This completes the durable path: **real volume created → pod attaches GPU → model runs → output validated → patent packet completed**. If the provider had more quirks (e.g. attachment must precede labeling), Brudo would catch them either by repeated discovery queries or by handling new error messages.

Key insight: Brudo now treats the cloud provider as the authority on API shape. Any invalid request is reinterpreted by querying the provider’s schema/endpoints, not by asking Codex. This aligns with best practices of agent legibility and strict interfaces [8†L379-L382] [8†L383-L392]. In short, we turned the one-off Codex “repair” step into a deterministic, testable routine. The result is a robust, minimal deployment pipeline that no longer hard-codes assumptions about the provider’s API.

Sources: Official API docs and REST best practices show that HTTP 400 indicates bad request data [15†L188-L191]; Kubernetes examples show PV failures on schema mismatch [1†L239-L247]. Agent-first engineering literature emphasizes making external contracts explicit and parsed by code [8†L379-L382] [8†L383-L392]. These guided the above solution.
